PRODUCTION ARCHITECTURE REVIEW

Kumbh Tent Booking Platform

Reverse-flow calculation and backend module mapping — the customer journey traced backward from a purged, closed-out booking to the OTP screen that started it, against the live Go microservice backend.

Customer | Admin / Ops | Payments | Notifications | Compliance

Archive → Refunds → Payment → Booking → Discovery → **Registration**

· 24 stages, 6 phases, traced against 5 live Go services

Document purpose

A production-oriented review of the booking lifecycle, read backward from outcome to origin.

1

Reverse walkthrough

Every stage from archival cleanup back to registration — what happens and why, in the order a completed stay actually resolves.

2

Backend module mapping

Each stage tied to the exact file inside `kumbh_backend/` that owns it — `auth-service`, `tent-service`, `booking-service`, `payment-service`, `api-gateway`.

3

Cross-cutting systems

Identity, RBAC, rate limiting, notifications, distributed locking — the modules that run underneath every stage, not tied to one step.

4

System architecture

How the Flutter customer app reaches the four services through CloudFront and the gateway, down to Postgres, S3 and Razorpay.

The forward flow, at a glance

24 stages, grouped into 6 phases — the spine this review runs backward across.

STAGES 1–4
Identity
Registration to OTP session

STAGES 5–7
Discovery
Browse, tent detail, reviews

STAGES 8–12
Booking & inventory
Pricing, unit hold, reference

STAGES 13–16
Payment & confirm
Order, signature, transaction

STAGES 17–21
Stay & reminders
Check-in, refunds, cron

STAGES 22–24
Closeout
No-show, review, purge

Why calculate it in reverse?

FORWARD VIEW

Builds the guest journey

- Shows how a pilgrim moves from OTP login to a checked-out stay
- Good for onboarding a new engineer to the product
- Answers: “what happens next?”

REVERSE VIEW

Validates the system that runs it

- Starts from a purged, closed booking and asks what had to be true before it
- Surfaces every dependency: no purge without a settled refund, no settled refund without a signed payment
- Answers: “what backend module makes this real?”

Status — what shipped this engagement

Every stage on the following pages carries its build status. Unlike a first pass, there is no active backlog blocking any stage below — the watchlist tracks operational follow-ups, not open bugs.

- Live in production
- Operational follow-up

OPEN FOLLOW-UPS (NOT STAGE-BLOCKING)

SMS provider — OTP is still delivered via a debug flag on the API response rather than a real gateway; `OTP_DEBUG` must go back to `false` once one is wired up.

Custom domain — the API currently terminates TLS at a CloudFront distribution as a stopgap; an ALB + ACM certificate is the target once a domain is registered.

Credential rotation — the Razorpay test key pair used during development should be rotated in the Razorpay dashboard.

Reverse walkthrough

Working backward from where a stay finally resolves.

`Stages 24–21` **Closeout & Archival** *Where a stay finally ends*

24 **Retention purge** Live `reminders.go`

A cancelled booking older than 30 days is deleted on the nightly sweep — unless a refund against it is still unsettled, in which case it's retained so the customer and staff never lose visibility of money still owed.

`Daily cron` → `refunds.status check` → `PostgreSQL DELETE`

23 **Guest review & rating** Live `tent_reviews.go`

Accepted once a booking has moved past payment — `confirmed`, `checked_in` or `completed` — not only after the nightly job marks it done, so the realistic moment a guest actually rates their stay is covered.

`canReviewBookingStatus()` → `reviews table` → `tent rating recompute`

22 **Booking auto-completion** Live `reminders.go`

The 08:00 IST job flips any `confirmed` booking whose checkout date has passed to `completed`, and pushes a "thank you for staying" notification.

`runIfLeader("reminder_scheduler")` → `UPDATE bookings` → `FCM`

21 **No-show sweep** Live noshow.go

A booking still unpaid or unchecked-in past its tent's configured cutoff hour is marked no_show, releasing the held unit.

15-min cron → cutoff comparison → status transition

Stages 20–17 Stay, Reminders & Refunds

The active phase, and what happens when plans change

20 **Distributed cron lock** Live cronlock.go

Every scheduled job — reminders, refund retry, no-show sweep, news poll — runs behind a Postgres session advisory lock, so the 4 EC2 instances behind the load balancer don't each run the same job on their own clock.

pg_try_advisory_lock → one winner per tick

19 **Refund settlement & retry** Live refunds.go

A tiered cancellation policy quotes what's owed from what was actually collected, not the booking total. The gateway call happens after the transaction commits — a slow refund API never holds a row lock — and a failure is retried, never silently dropped.

Tiered policy → Razorpay refund API → retry worker

18 **Cancellation** Live handler.go

The booking row is locked FOR UPDATE so two concurrent cancel taps can't both create a refund; a unique index on refunds.booking_ref makes a second one impossible even if the lock were bypassed.

FOR UPDATE → refund quote → refunds INSERT

17 **Check-in & pre-arrival reminders** Live notification_jobs.go

Front-desk check-in is a single admin call; the day-before and day-of reminders (and the pay-at-tent nudge) are each deduplicated by a unique (phone, type, ref_key) row, so a scheduler re-run never repeats a push.

adminCheckInBooking → notifyUser() → notification_log

Stages 16–13 Payment & Confirmation

Where money actually moves

16 **Duplicate-payment guard** Live `016_payment_integrity.sql`

A unique partial index on `payments.razorpay_payment` for successful rows means the same Razorpay payment ID can never be recorded twice, even under a retried client call or a webhook race.

`UNIQUE INDEX` → `idempotent 200 on retry`

15 **Signature verification & transactional confirm** Live `payment-service/handler.go`

The HMAC signature is checked against the order + payment ID, then the payment record and the booking's `pending → confirmed` transition happen inside one database transaction — either both land, or neither does.

`HMAC-SHA256` → `db.Begin()` → `atomic commit`

14 **Coupon redemption** Live `payment-service/handler.go`

`used_count` increments only once a `pending` booking actually flips to `confirmed` — an abandoned, never-paid booking no longer burns a coupon use forever.

`RowsAffected > 0` → `UPDATE coupons`

13 **Razorpay order creation** Live `orders.go`

The paise amount is rounded, not truncated — a total that floats to one paisa under its true value no longer opens a Razorpay order short of the booking's actual price.

`math.Round(total×100)` → `Razorpay Orders API`

Stages 12–8 Booking & Inventory

Turning a search result into a held unit

12 **Server-side price quote** Live `pricing.go`

Bed surcharge, child fee and the GST slab (12% at ≤ ₹7,500/night, 18% above — decided by the nightly tariff, never the aggregated stay total) are computed server-side and unit-tested, never trusted from the client.

`computeBookingCharges()` → `taxable amount` → `total`

11 **Children-bracket cap** Live `pricing.go · booking_form_screen.dart`

Both the under-5 and 5–12 brackets are capped at 2 per booking, enforced in the Flutter counters and independently clamped server-side, so a tampered request can't claim more than the form allows.

client cap → server clamp

10 **Per-night unit hold** Live handler.go · createBooking

The tent row is locked FOR UPDATE and every night in the requested range is checked against booked units — one bad night in the middle of a range rejects the whole booking, not just that date.

SELECT … FOR UPDATE → dailyAvailability() → reject or hold

9 **Booking reference** Live handler.go

A 7-digit crypto-random reference (10M possibilities) is generated and retried — re-locking the tent row — on the rare collision, instead of a raw failure the first time one happened.

crypto/rand → INSERT → retry on 23505

8 **Live availability** Live tent-service/handler.go

Booked units are summed per tent (SUM(units)), not counted per row — a multi-unit booking no longer reads as a single reservation and a tent can no longer be oversold.

SUM(b.units) → total_units − booked

Stages 7–5 Discovery

Where a pilgrim actually starts looking

7 **Browse & tent detail** Live tent-service/handler.go

Class-filtered catalog with cancellation policy surfaced up front; images now resolve to real, publicly-served S3 URLs matched to each tent's class rather than an empty array.

GET /tents → GET /tents/:id

6 **Guest reviews** Live tent_reviews.go

Reviewer phone numbers are masked before they ever leave the API; the tent's average rating recomputes on every new review.

reviews table → masked phone

5 **Favourites & snan reminders** Live favourites.go

Every read and write derives its owner from the caller's own verified token — a caller-supplied phone in the query string or body is accepted but ignored, closing off reading or clearing someone else's saved list.

`verifiedPhone(c)` → `favourites / snan_reminders`

Stages 4–1 **Identity** *Where every journey actually begins*

4 **Admin session** Live `auth-service/handler.go`

Bcrypt-verified password issues a role-carrying JWT; login attempts are rate-limited by both username and caller IP, backed by a DB counter shared across every instance, not an in-memory one.

`bcrypt` → `rateLimitExceeded()` → `JWT`

3 **OTP verify** Live `security.go`

A crypto-random 6-digit code, rate-limited on both send and verify. The response body echoing the code back is a deliberate, flagged debug aid (`OTP_DEBUG`) for a build with no SMS provider wired up yet — never the default.

`secureNumericCode(6)` → `otp_store` → `JWT on match`

2 **Identity carried forward** Live `identity.go — every service`

Every service verifies the caller's own JWT independently rather than trusting an `X-User-Phone` header forwarded by the gateway — a request that reached a service directly could no longer impersonate any phone number it liked.

`verifiedPhone(c)` → `signature check per-service`

1 **Registration** Live `login_screen.dart`

Phone number captured; token and phone are written to `FlutterSecureStorage`, not `SharedPreferences` — the one place every later screen (payment included) reads identity from.

`POST /auth/send-otp` → `FlutterSecureStorage`

Cross-cutting modules

Running underneath every stage above — not tied to a single step in the flow.

`identity.go`

Verified identity

`rbac.go`

Admin RBAC

Each service decodes and checks the caller's own JWT rather than trusting a forwarded header.

`requireAdmin()` gates every `/admin/*` route independently in each service, not only at the gateway.

`ratelimit.go`

Rate limiting

A DB-backed counter table, so the limit holds across all 4 instances behind the load balancer.

`reminders.go`

Notifications

Broadcast pushes fan out over a bounded 20-worker pool; every scheduled push is deduplicated per user.

`cronlock.go`

Distributed locking

A Postgres advisory lock ensures a scheduled job runs once per tick, not once per EC2 instance.

`shutdown.go`

Graceful shutdown

SIGTERM drains in-flight requests for up to 15s before the process exits, on every service.

`admin_audit_log`

Audit trail

Who changed a tent, coupon or booking status, and when — every admin mutation writes through one table.

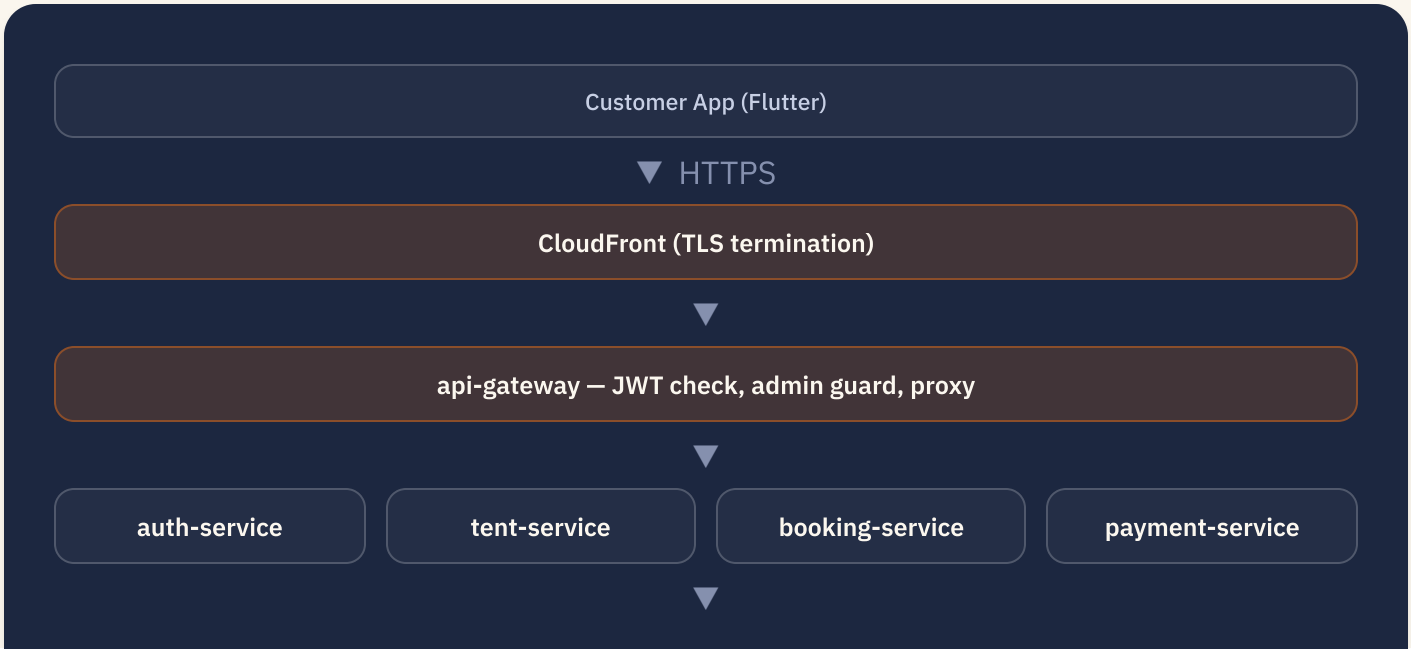
`CloudFront`

TLS termination

The customer app talks HTTPS to CloudFront; the hop to the EB origin stays inside AWS's own network.

Backend architecture flow

One gateway in front of four independent Go services, each owning its own slice of the booking lifecycle.



RDS PostgreSQL	S3 (tent images, releases)	Razorpay	Firebase Cloud Messaging

Every request below the gateway is reachable only after `identity.go` and, for admin routes, `rbac.go` — auth is re-verified at each service, not assumed from the hop before it.

Recommended next steps

1 **Rotate the Razorpay test credentials**

The development key pair was exposed in early commits; rotate it in the Razorpay dashboard independently of any code change.

OWNER — PAYMENTS

2 **Move from CloudFront to ALB + ACM**

CloudFront is a working stopgap for TLS; a registered domain plus an ACM certificate on an Application Load Balancer is the intended long-term front door.

OWNER — INFRA

3 **Wire a real SMS provider**

Once OTPs deliver over SMS, `OTP_DEBUG` goes back to `false` and the code stops appearing in the API response.

OWNER — AUTH

4 **Scrub leaked credentials from git history**

The old key pair still exists in earlier commits even after rotation; a history rewrite is a deliberate, separately-scheduled action given its blast radius.

OWNER — PLATFORM